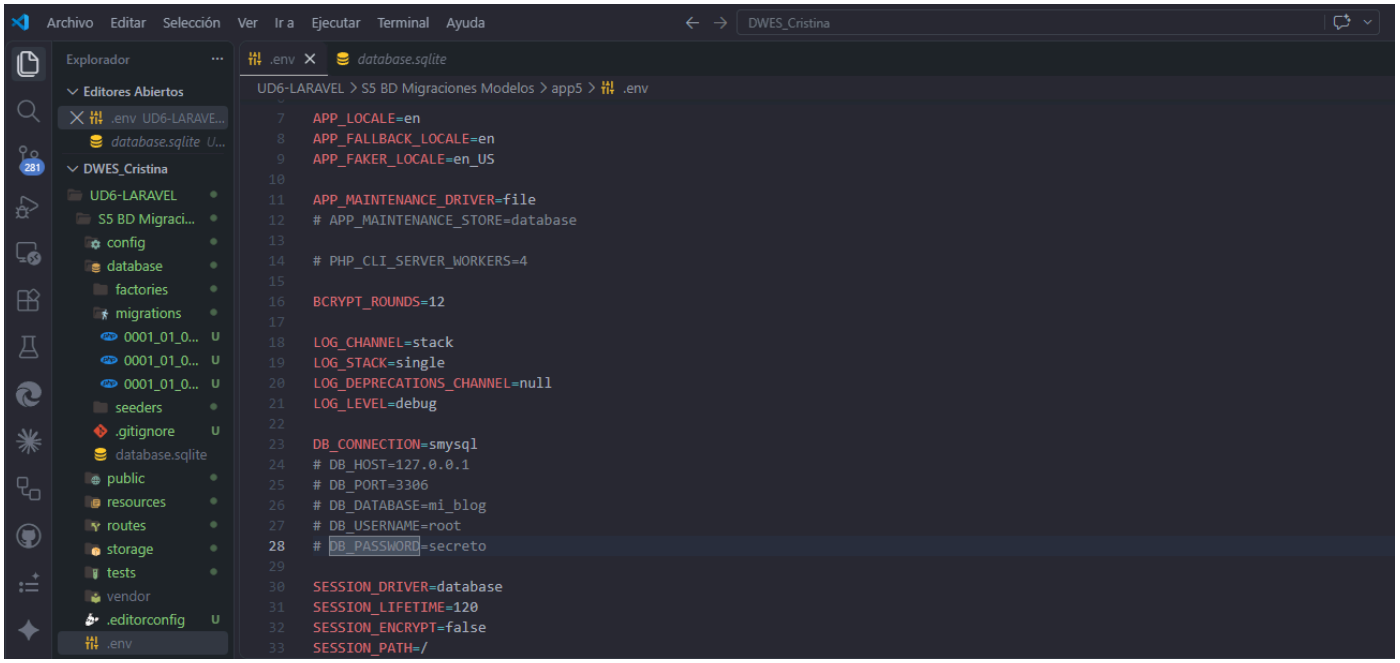
Seseion 5

Creamos un nuevo proyecto y dentro del punto .env cambiamos DB_CONNECTION, este apartado sirve para configurar nuestra BBDD.



Migraciones

Una migracion contiene dos metodos up() y down().

.env database.sqlite 0001_01_01_000000_create_users_table.php U

UD6-LARAVEL > S5 BD Migraciones Modelos > app5 > database > migrations > 0001_01_01_000000

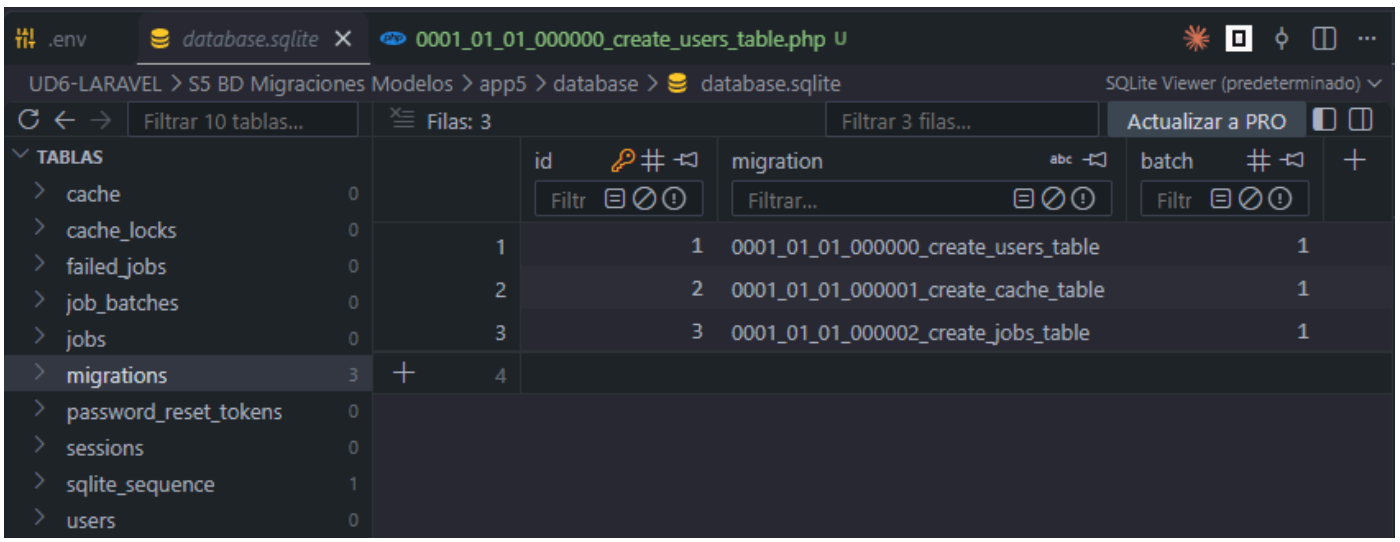
```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });
```

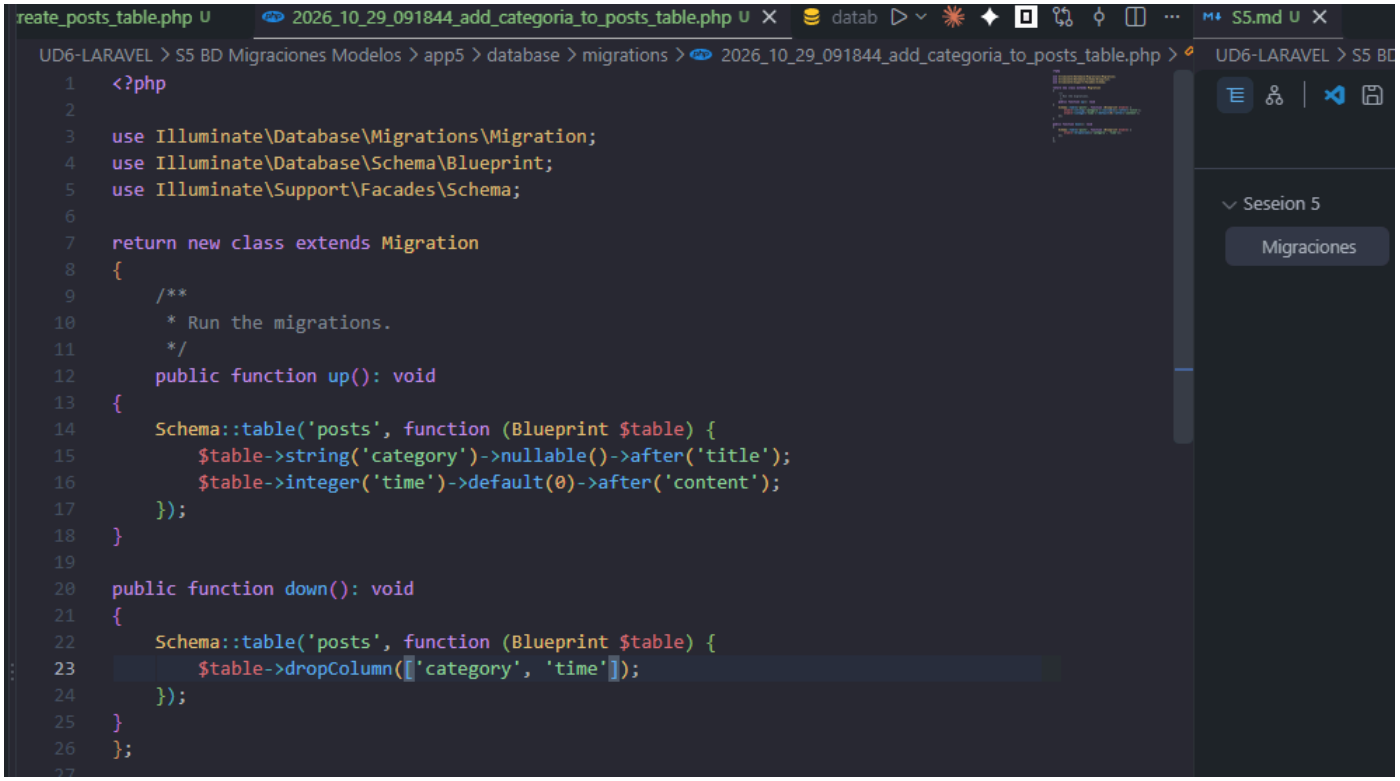
```php
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

Laravel lleva un registro de las migraciones que se han ejecutado.



Modificar tablas existentes

- php artisan make:migration add_(nombre tabla)



Cambiar el tipo de columna

Renombrar y eleminar columnas

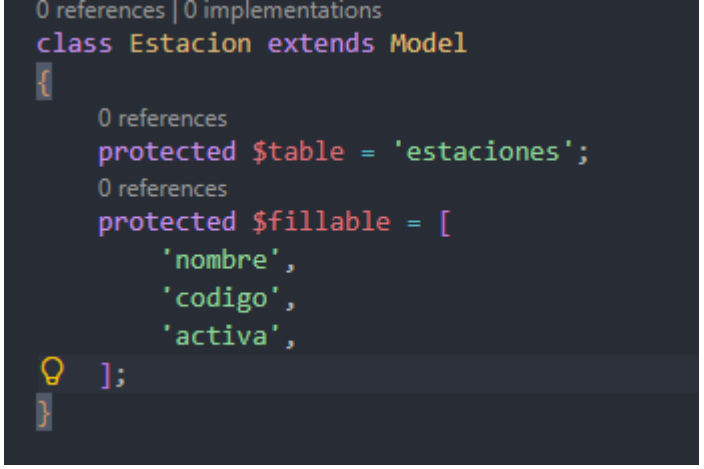
Modelos

Podemos crear modelos usando el comando:

'php artisan make:model Estacion'

Opciones que se pueden añadir (-m migraciones, -f genera una factoria o -s que crea un seeder)

Asignación masiva y `$fillable`



```
0 references | 0 implementations
class Estacion extends Model
{
    0 references
    protected $table = 'estaciones';
    0 references
    protected $fillable = [
        'nombre',
        'codigo',
        'activa',
    ];
}
```

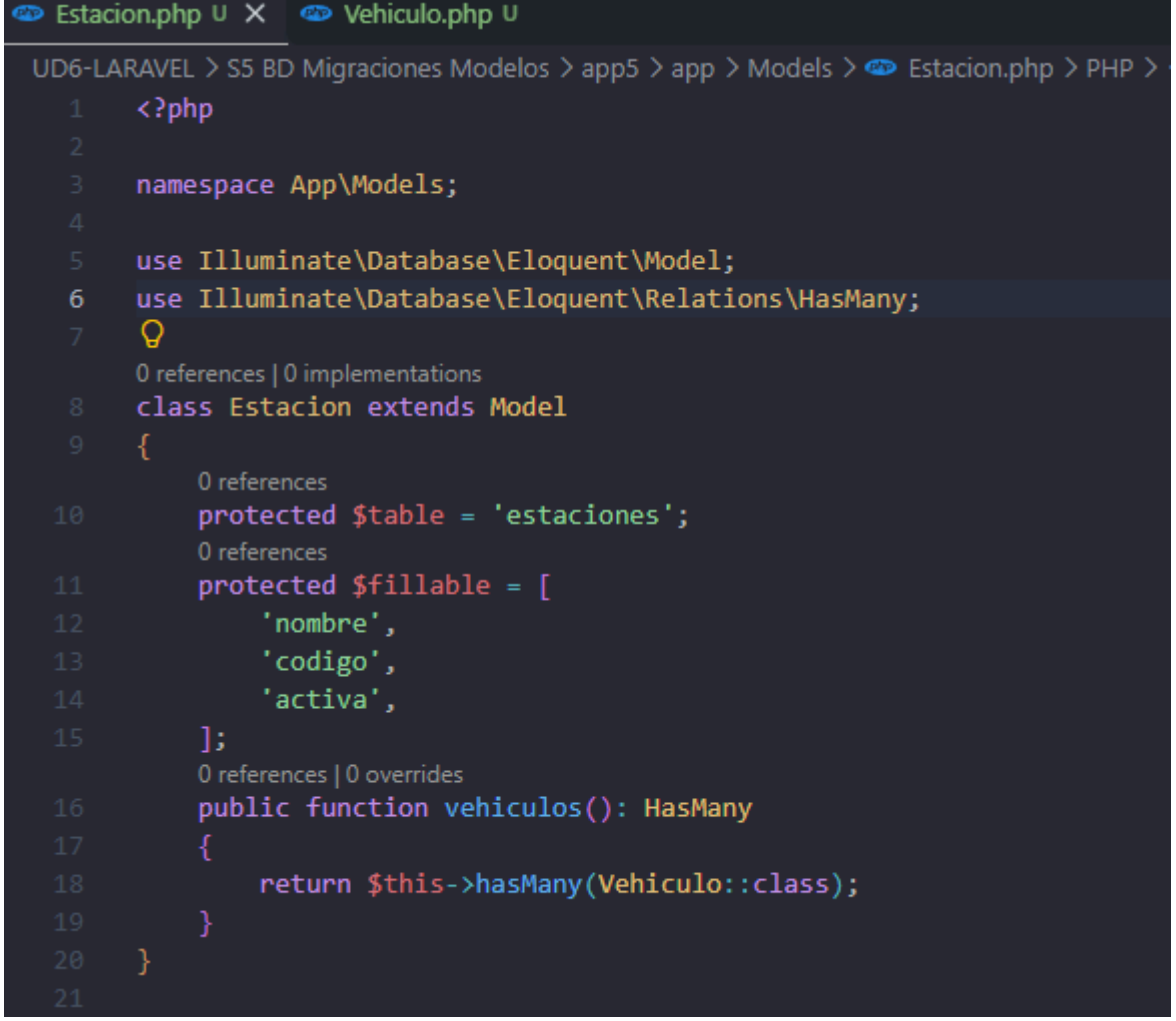


```
public function up(): void
{
    Schema::create('estaciones', function (Blueprint $table) {
        $table->primary('codigo');
        $table->string('nombre');
        $table->boolean('activa')->default(true);
        $table->timestamps();
    });
}
```

La alternativa `$guarded` define campos protegidos.

Relaciones entre modelos

Una clave foránea relaciona registros en la base de datos.



Las funciones de relacion estaran dentro del modelo, esta funcion nos dara una clase vehiculo.

Relación inversa: `belongsTo`

UD6-LARAVEL > S5 BD Migraciones Modelos > app5 > app > Models > Vehiculo.php > PHP Intelep

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

1 reference | 0 implementations
class Vehiculo extends Model
{
    0 references
    protected $table = 'vehiculos';
    0 references
    protected $primaryKey = 'codigo';

    0 references
    protected $fillable = [
    'estacion_id',
    'modelo',
    'tipo',
    'disponible',
];
0 references | 0 overrides
public function estacion(): BelongsTo
    {
        return $this->belongsTo(Estacion::class);
    }
}
```

`belongsTo()` indica que la tabla actual (`vehiculos`) contiene la clave foránea que apunta al modelo relacionado (`Estacion`).

Proyecto autoevaluable.

Hemos creado la migracion --> Requiere revision para ello usamos primero el comando 'php artisan make:migration add_requiere_revision_to_vehiculos_table'

Actualizamos up() y down().

```php
public function up(): void
    {
        Schema::table('vehiculos', function (Blueprint $table) {
            $table->boolean('requiere_revision')->default(false);
        });
    }

    /**
     * Reverse the migrations.
     */
```

```php
    public function down(): void
    {
        Schema::table('vehiculos', function (Blueprint $table) {
            $table->dropColumn('requiere_revision');
        });
    }
```

Actualizamos el $fillable de vehiculo.

```php
    0 references
    protected $fillable = [
    'estacion_id',
    'modelo',
    'tipo',
    'disponible',
    'autonomia_km',
    'requiere_revision',
];
```

Usamos tinker para crear los nuevos datos que queremos guardar para que luego los mostremos en la vista.

Por ultimo, cambiamos la vista para poder ver los cambios.

index.blade.php U | Estaciones de movilidad | Vehiculo.php U

UD6-LARAVEL > S5 BD Migraciones Modelos > app5 > resources > views > estaciones > index.blade.php > html > body

```
2   <html lang="es">
15  <body>
19          <article>
20              <h2>{{ $estacion->nombre }}</h2>
21              <p>Código: {{ $estacion->codigo }}</p>
22              <p class="{{ $estacion->activa ? 'estado' : 'estado inactivo' }}">
23                  {{ $estacion->activa ? 'Activa' : 'Inactiva' }}
24              </p>
25              <h3>Vehículos</h3>
26              <ul>
27                  @forelse ($estacion->vehiculos as $vehiculo)
28                      <li>
29                          {{ $vehiculo->modelo }} · {{ $vehiculo->tipo }} ·
30                          {{ $vehiculo->disponible ? 'Disponible' : 'No disponible' }} ·
31                          Autonomía: {{ $vehiculo->autonomia_km ?? 'Sin indicar' }} km
32                          @if ($vehiculo->requiere_revision)
33                                  · <strong class="inactivo">Requiere revisión</strong>
34                          @endif
35                      </li>
36                  @empty
37                      <li>Esta estación todavía no tiene vehículos.</li>
38                  @endforelse
39              </ul>
40          </article>
41      @empty
42          <p>No hay estaciones registradas.</p>
43      @endforelse
44  </body>
45  </html>
```

Estaciones de movilidad

Estación de prueba

Código: TEST-01

Activa

Vehículos

- Bicicleta urbana · bicicleta · Disponible · Autonomía: Sin indicar km
- Patinete eléctrico · patinete · No disponible · Autonomía: 25 km · **Requiere revisión**